

UNIT – 4

MEMORY MANAGEMENT

- 4.1 Describe briefly address binding, dynamic loading, dynamic linking
- 4.2 Define overlays
- 4.3 Describe briefly on swapping
- 4.4 Explain single partition allocation
- 4.5 Explain multiple partition allocation
- 4.6 Explain the concept of fragmentation
- 4.7 Explain paging concept
- 4.8 Explain how logical address is translated into physical address
- 4.9 Explain segmentation and segmentation with paging
- 4.10 Define virtual memory techniques
- 4.11 Describe demand paging
- 4.12 Describe page replacement
- 4.13 Discuss on page replacement algorithms
 - 4.13.1 FIFO
 - 4.13.2 LRU
 - 4.13.3 Optimal
- 4.14 Explain the concept of thrashing
- 4.15 Explain working set model and page fault frequency

Introduction to Memory Management

Memory

Computer memory can be defined as a collection of some data represented in the binary format. On the basis of various functions, memory can be classified into various categories. A computer device that is capable to store any information or data temporally or permanently is called storage device.

There are different types of memory in a computer system:

- **Primary Memory** (e.g., RAM, cache): Fast, volatile memory used for current processes and data.
- **Secondary Memory** (e.g., hard drives, SSDs): Slower, non-volatile storage used for long-term data.
- **Virtual Memory**: A section of the hard drive used as an extension of RAM when it's full.

Memory management

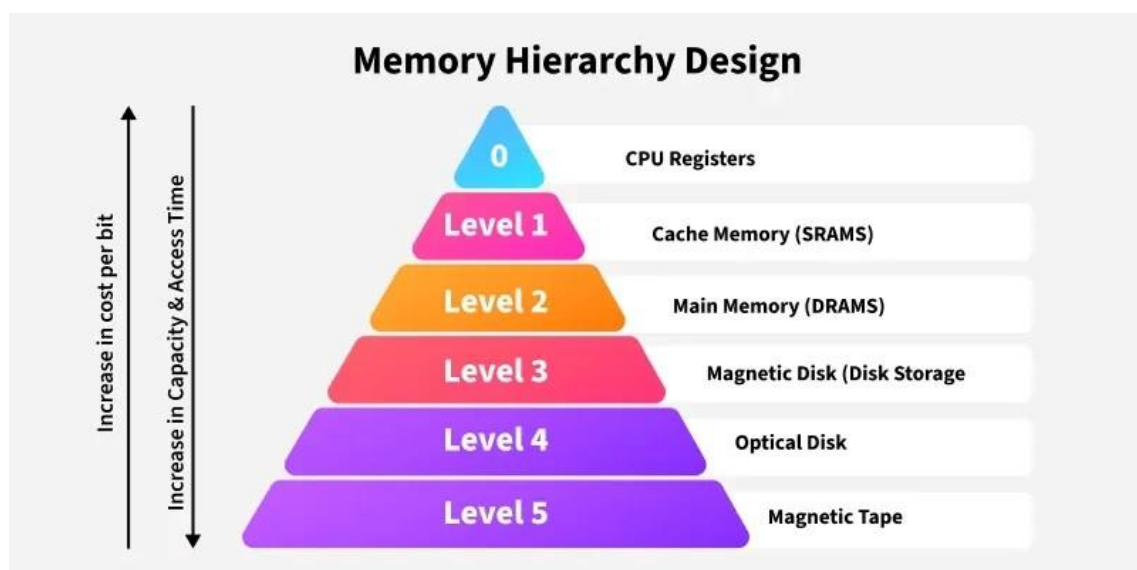
The term Memory can be defined as a collection of data in a specific format. It is used to store instructions and processed data.

The memory comprises a large array or group of words or bytes, each with its own location.

The primary motive of a computer system is to execute programs. These programs, along with the information they access, should be in the main memory during execution.

The CPU fetches instructions from memory according to the value of the program counter.

Memory Hierarchy



In the Computer System Design, Memory Hierarchy is an enhancement to organize the memory such that it can minimize the access time. } The Memory Hierarchy was developed based on a program behaviour known as locality of references. } The figure below clearly demonstrates the different levels of memory hierarchy :

1. Registers

Registers are the fastest and smallest memory located directly within the CPU. They store temporary data such as operands and instruction addresses during processing. For example, the ACC (Accumulator) register holds the result of an arithmetic operation in a microprocessor.

2. Cache Memory

Cache memory stores frequently accessed data to speed up CPU operations by reducing access time to main memory. It is usually split into L1, L2, and L3 levels. For example, when you repeatedly access the same webpage, cache helps load it faster by storing parts of it temporarily.

3. Main Memory (RAM)

Main memory holds data and programs currently being used by the system. It is larger than cache but slower, and it loses its contents when power is off. For example, when editing a Word document, the file resides in RAM until saved.

4. Secondary Storage

Secondary storage provides permanent data storage and is non-volatile. It includes devices like hard drives and SSDs. For example, your computer's operating system is stored on an SSD and loaded into RAM during startup.

5. Magnetic Disk

Magnetic disks, such as traditional hard disk drives (HDDs), use magnetic storage to store data permanently. They are slower than SSDs but often used for bulk data storage due to lower cost. For example, old desktop PCs use HDDs to store videos, photos, and files.

Optical Disk

Optical disks are a type of secondary storage that use laser technology to read and write data. Common types include CDs, DVDs, and Blu-ray discs, which are typically used for media, software distribution, or backups. For example, a DVD might store a movie or a large software installation like Microsoft Office.

Characteristics Of Memory Hierarchy:

1. Speed

Higher levels of memory like registers and cache are extremely fast, allowing quick access to data. As you move down the hierarchy, memory becomes slower. For example, accessing data from an HDD is much slower than from L1 cache.

2. Cost per Bit

Faster memory is more expensive per bit due to advanced technology and limited size. Slower memory types are cheaper and used for larger storage. For instance, RAM is more costly than a hard drive per gigabyte.

3. Capacity

As memory speed decreases, its storage capacity increases. Registers hold only a few bytes, while secondary storage can hold terabytes. For example, a typical laptop may have 16GB RAM and 1TB HDD.

4. Volatility

Top-level memory (like registers and RAM) is volatile and loses data when power is off. Lower-level storage (like HDDs and SSDs) is non-volatile and retains data. For example, files on a USB drive remain saved after shutdown.

5. Access Frequency

Frequently accessed data is kept in faster memory levels to speed up processing. Less-used data is stored in slower, larger memory. For example, a browser's cache stores website data to load pages quickly.

6. OS Management

The operating system manages the memory hierarchy by moving data between levels as needed. It uses techniques like paging and caching to optimize performance. For example, Windows may use virtual memory when RAM is full.

Advantages of Memory Hierarchy

1. Improves system performance by keeping frequently used data closer to the CPU.
2. Reduces average memory access time with the use of fast upper-level memory.
3. Balances cost and efficiency by combining different types of memory.

Disadvantages of Memory Hierarchy

1. Complex design and implementation increase system development costs.
2. Managing data between levels adds overhead to the operating system.
3. Cache misses or page faults can cause performance delays.

Address Binding, Dynamic Loading, Dynamic Linking

Address Binding

Address binding is the process of mapping program addresses (logical or symbolic addresses) to physical memory addresses. It can happen at three stages: compile-time, load-time, or run-time. For example, in compile-time binding, the compiler assigns fixed memory addresses before execution.

Here are the types of address binding explained briefly:

1. Compile-Time Binding

In compile-time binding, the compiler assigns physical memory addresses to program instructions and data before execution. The program must be loaded at that fixed address in memory to run correctly. If the program is moved, it must be recompiled.

Example: Early simple systems where programs run from fixed memory locations.

2. Load-Time Binding

Load-time binding occurs when the OS loader assigns physical addresses to the program during the loading process into memory. The program can be loaded at any address, but once loaded, its address remains fixed during execution.

Example: When you load a program into RAM, the loader maps logical addresses to physical ones.

3. Execution-Time (Run-Time) Binding

In execution-time binding, the mapping from logical to physical addresses is done dynamically during program execution, usually with the help of hardware like the Memory Management Unit (MMU). This allows programs to be moved in memory during execution.

Example: Systems using virtual memory where address translation happens on the fly.

Dynamic Loading

Dynamic loading is a technique where a program loads specific modules or routines into memory only when they are needed during execution. This reduces the initial memory usage and makes the program more efficient by avoiding the loading of unused code. It also allows faster startup and better resource management. The operating system handles the loading process when the function is first called.

Real-time example: In a media player, the subtitle feature module is loaded only when a user enables subtitles during video playback.

Dynamic Linking

Dynamic linking is the process of linking external libraries to a program during runtime instead of at compile time. This allows multiple programs to share the same version of a library, reducing memory usage and disk space. It also makes updating libraries easier.

without recompiling dependent applications. The operating system uses a dynamic linker/loader to manage these shared libraries.

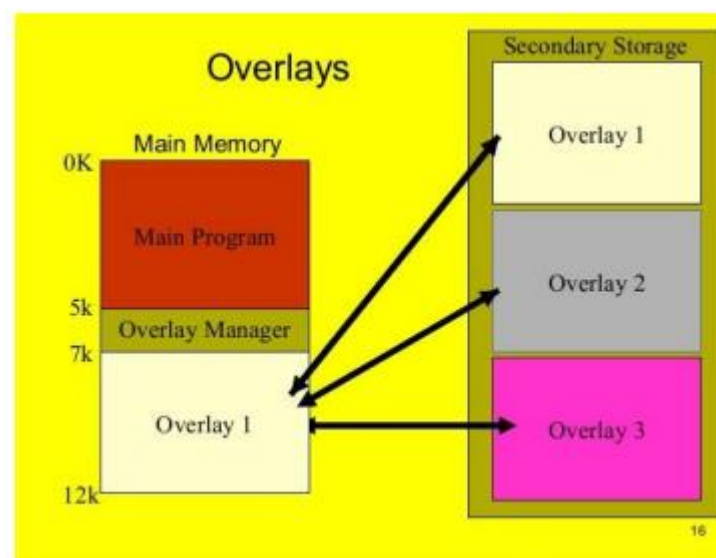
Real-time example: A Windows application dynamically links to `msvcrt.dll` (Microsoft C Runtime Library) to perform standard input/output operations.

Overlay in Memory Management

Overlay is a memory management technique used to run programs larger than the available main memory by loading only necessary parts of the program into memory at a time. The rest of the program remains on secondary storage (like a disk) and is loaded as needed, replacing other parts (overlays) in memory.

Key Points:

- It breaks a large program into smaller modules (overlays).
- Only one or a few overlays are loaded at a time.
- It requires manual programming effort to manage overlays.
- Commonly used in systems with limited memory (e.g., early embedded systems).



Advantages of Overlays in Memory Management

1. **Efficient Use of Limited Memory:** Allows large programs to run on systems with very small main memory.
2. **Memory Saving:** Only the required part of the program is loaded, saving RAM space.
3. **Better Control:** Programmers can manually optimize memory usage by deciding which modules load when.

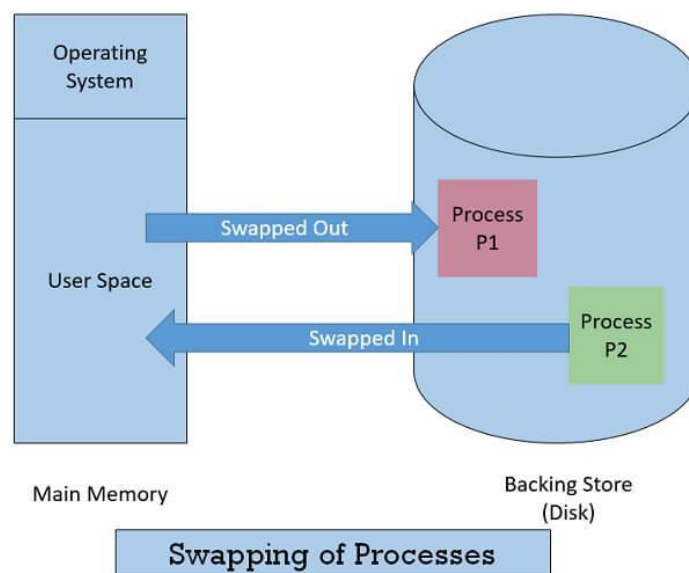
Disadvantages of Overlays in Memory Management

1. **Manual Complexity:** Programmers must manually divide and manage overlays, increasing development time and errors.

2. **Slower Execution:** Loading overlays during execution can cause delays due to frequent I/O operations.
3. **No True Multitasking:** Only one overlay is in memory at a time, limiting concurrent execution.

Swapping

- Swapping is a memory management technique and is used to temporarily remove the inactive programs from the main memory of the computer system.
- Any process must be in the memory for its execution, but can be swapped temporarily out of memory to a backing store and then again brought back into the memory to complete its execution. Swapping is done so that other processes get memory for their execution.



How Swapping Works:

1. The operating system selects a low-priority or idle process in memory.
2. The entire process is copied to a swap space on disk (also called the swap file or paging file).
3. This frees up RAM for other processes that need to run.
4. When the swapped-out process needs to run again, it is swapped back into RAM, possibly replacing another process.

Advantages of Swapping

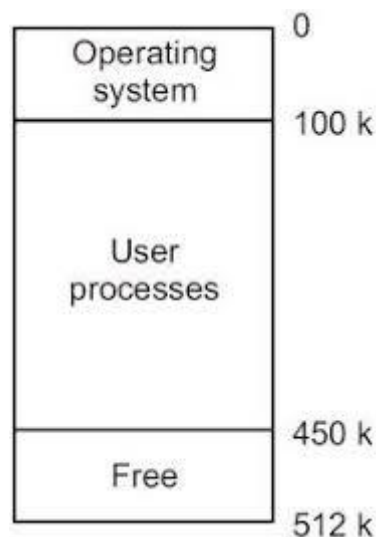
1. **Improves CPU Utilization:** Keeps the CPU busy by freeing memory for processes that need to run immediately.
2. **Flexibility:** Enables processes larger than available physical memory to execute by swapping parts in and out.
3. **Better Memory Management:** Helps avoid memory fragmentation by freeing contiguous memory space.

Disadvantages of Swapping

1. **Slow Process:** Swapping involves moving entire processes to disk and back, which is time-consuming.
2. **High I/O Overhead:** Frequent swapping causes excessive disk reads/writes, leading to performance degradation (thrashing).
3. **Wasted CPU Cycles:** CPU might be idle while waiting for swap operations to complete.

Single Partition Allocation

- In this type of allocation, relocation-register scheme is used to protect user processes from each other, and from changing operating-system code and data.
- Relocation register contains value of smallest physical address whereas limit register contains range of logical addresses. Each logical address must be less than the limit register.
- In this memory allocation method, the operating system resides in the low memory. The remaining memory treated as a single partition. This single partition is available for userspace. Only one job can be loaded in this user space.



Key Points:

- Memory is divided into one large partition.
- Only one process is loaded and executed at a time.
- There is no multitasking or sharing of memory between processes.
- The OS and the single process share the memory space.
- Commonly used in very early or simple systems.

Advantages:

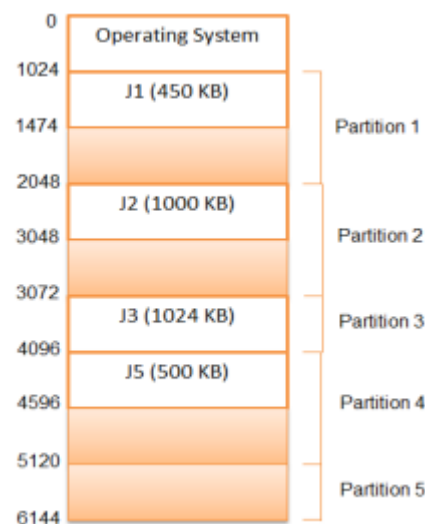
Simple to implement and manage, with no overhead of multitasking or complex memory allocation.

Disadvantages:

Very inefficient for modern systems since only one process can run at a time, wasting CPU and memory resources.

Multiple Partitioned Allocation

Multiple Partitioned Allocation divides the main memory into several fixed or variable-sized partitions, allowing multiple processes to be loaded and run concurrently. Each partition can hold exactly one process, enabling **multiprogramming** and better utilization of memory compared to single partition allocation.



This method can be implemented in 3 ways.

- Fixed-equal Multiple Partitions Memory Management
- .Fixed-variable Partitions Memory Management
- Dynamic Partitions Memory Management

Fixed-equal Multiple Partitions Memory Management:

In this memory management, the operating system occupies the low memory and the rest of the main memory is available for user-space. The user space divides into fixed partitions. The partition sizes are depending on the operating system.

Fixed-variable Partitions Memory Management:

In this memory management, the user space of the main memory is divided into the number of partitions. But the partition sizes are different lengths. The operating system keeps a table indicating which partitions of memory are available and which are occupied.

Dynamic Partitions Memory Management:

To eliminate some of the problems with fixed partitions, an approach known as Dynamic Partitions. In this method, partitions are created dynamically. So that each process loads into a partition of the same size at that process.

Key Points:

- Memory is split into multiple partitions (fixed or dynamic sizes).
- Each partition can hold one process at a time.
- Supports multitasking by running multiple processes simultaneously.
- Requires memory management to allocate and deallocate partitions efficiently.
- Can suffer from fragmentation, especially with fixed partitions.

Advantages:

Improves CPU utilization by running multiple processes, enables multitasking, and uses memory more efficiently than single partitioning.

Disadvantages:

Can lead to internal or external fragmentation and requires more complex memory management.

Fragmentation

Fragmentation is a condition in which storage space is used inefficiently, reducing the performance and memory utilization of a system. It occurs in memory management when free memory is divided into small pieces, making it hard to allocate space for new processes, even if enough total memory exists.

Types of Fragmentation:

1. Internal Fragmentation

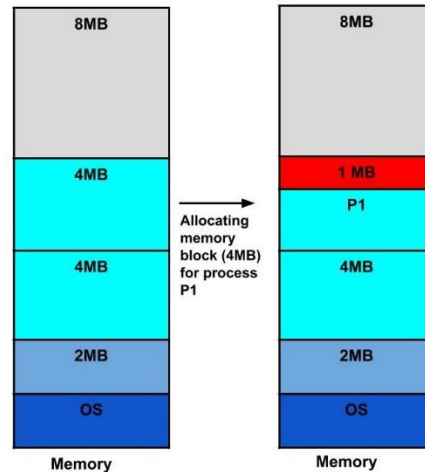
- Happens when memory is allocated in fixed-sized blocks, and the process doesn't use the entire block.
- The unused space inside the allocated block is wasted.
- ✨ *Example:* Allocating 100 KB block to a process needing only 80 KB wastes 20 KB.

2. External Fragmentation

- Occurs when free memory is scattered in small blocks throughout the system.
- Even if enough total memory is available, it is not contiguous, so large processes can't be allocated.
- ✨ *Example:* 10 free blocks of 4 KB scattered in memory can't satisfy a 20 KB process.

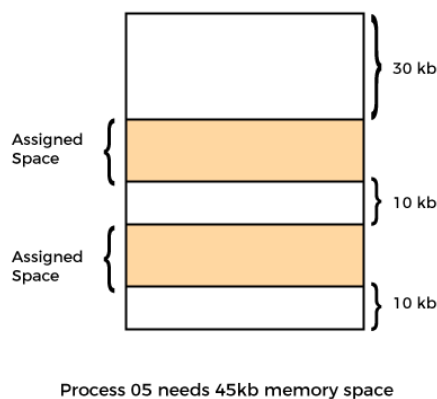
For Example:

Assume that memory allocation in RAM is done using fixed partitioning (i.e., memory blocks of fixed sizes). 2MB, 4MB, 4MB, and 8MB are the available sizes. The Operating System uses a part of this RAM.



Let's suppose a process P1 with a size of 3MB arrives and is given a memory block of 4MB. As a result, the 1MB of free space in this block is unused and cannot be used to allocate memory to another process. It is known as internal fragmentation.

External fragmentation happens when a dynamic memory allocation method allocates some memory but leaves a small amount of memory unusable. The quantity of available memory is substantially reduced if there is too much external fragmentation. There is enough memory space to complete a request, but it is not contiguous. It's known as external fragmentation.



In Operating Systems (OS), First Fit, Best Fit, and Worst Fit are memory allocation algorithms used to allocate memory to processes. These algorithms determine which free block of memory is selected when a process needs to be loaded into memory.

First Fit:

The First Fit memory allocation strategy places a process into the first free memory block that is large enough to accommodate it. The operating system scans memory from the beginning and selects the first suitable space. This method is fast and simple to implement

because it stops searching as soon as it finds a fit. However, it can cause external fragmentation, as it leaves behind many small, unused spaces in memory.

Example: If blocks are 10 KB, 20 KB, 30 KB and a 15 KB process arrives, it will be placed in the 20 KB block.

Best Fit:

The Best Fit strategy looks at all available memory blocks and selects the one that is closest in size to the process's requirement — the “best” fitting block. This can help reduce internal fragmentation by minimizing wasted space inside allocated blocks. However, this method is slower, as it searches the entire memory, and it may increase external fragmentation due to many small leftover spaces.

Example: If a process needs 12 KB and free blocks are 13 KB, 18 KB, and 25 KB, it will be placed in the 13 KB block.

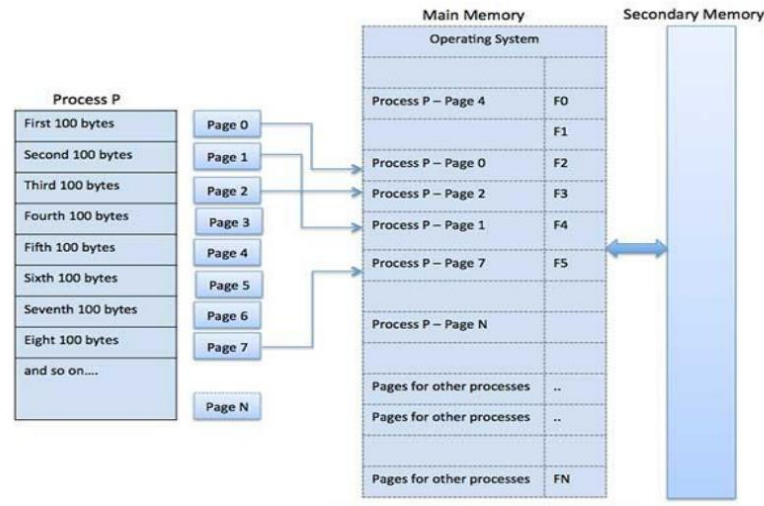
Worst Fit:

The Worst Fit strategy allocates memory from the largest available block. The idea is to leave behind a large enough fragment that can be reused for another process later. Although this approach may reduce small fragments, it can cause internal fragmentation and may not always be effective in keeping memory usable. It is also slower, as it needs to search the entire memory for the largest block.

✎ *Example:* If free blocks are 15 KB, 25 KB, and 35 KB, and a 12 KB process arrives, it will be allocated in the 35 KB block.

Paging Concept

- Paging is a memory management technique in which process address space is broken into blocks of the same size called pages (size is power of 2, between 512 bytes and 8192 bytes). The size of the process is measured in the number of pages.
- In Operating Systems, Paging is a storage mechanism used to retrieve processes from the secondary storage into the main memory in the form of pages.
- The main idea behind the paging is to divide each process in the form of pages. The main memory will also be divided in the form of frames.
- One page of the process is to be stored in one of the frames of the memory. The pages can be stored at the different locations of the memory but the priority is always to find the contiguous frames or holes.
- Pages of the process are brought into the main memory only when they are required otherwise they reside in the secondary storage.
- Different operating system defines different frame sizes. The sizes of each frame must be equal. Considering the fact that the pages are mapped to the frames in Paging, page size needs to be as same as frame size.



Paging is a memory management technique used by operating systems to avoid external fragmentation and efficiently manage RAM. In this method, both the logical memory (used by processes) and the physical memory (RAM) are divided into fixed-size blocks.

- The blocks in logical memory are called pages.
- The blocks in physical memory are called frames.
- When a program is loaded, its pages are placed into available frames in RAM, not necessarily in a continuous order.

Logical Address Is Translated Into Physical Address

Logical Address:

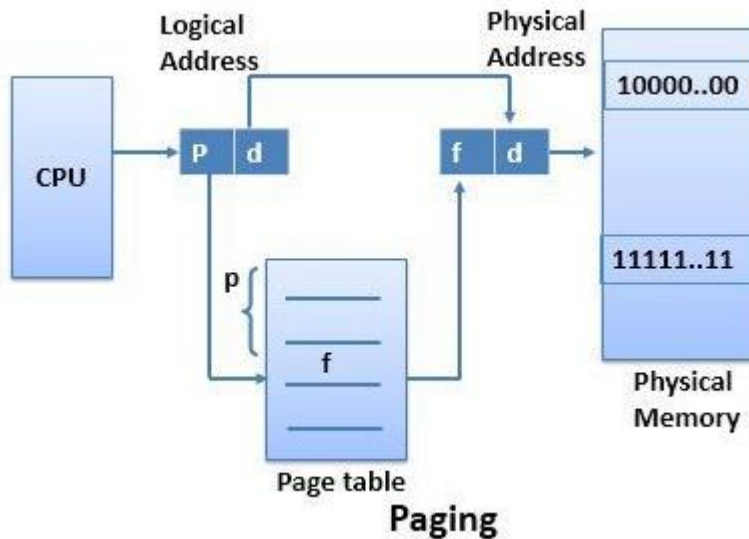
A logical address is the address generated by the CPU during program execution. It is also known as a virtual address. This address is used by a program to access memory locations and is part of the abstraction provided by the operating system to separate the program's view of memory from the actual physical memory. Logical addresses allow programs to use memory independently of the physical location in the hardware, enabling features like memory protection and virtual memory.

Logical Address = page number+ page offset

Physical Address:

A physical address is the actual location in the computer's main memory (RAM) hardware where data or instructions reside. It refers to the real address on the memory module used to access the stored information. When a program runs, the operating system and the memory management unit translate the logical address generated by the CPU into a corresponding physical address, allowing the CPU to fetch or store data in the correct place in physical memory.

physical Address = frame number+ page offset



1) **Generation of Logical Address:**

The CPU generates a logical (or virtual) address when a program accesses memory during execution.

2) **Memory Management Unit (MMU) Involvement:**

The logical address is sent to the MMU, a hardware component responsible for address translation.

3) **Address Breakdown:**

The MMU divides the logical address into parts—commonly a page number (or segment number) and an offset.

4) **Page/Segment Table Lookup:**

The MMU uses the page or segment number to find the corresponding physical frame or segment base address in the page or segment table maintained by the OS.

5) **Calculating Physical Address:**

The MMU combines the frame base address (from the table) with the offset to form the complete physical address.

6) **Accessing Physical Memory:**

The physical address is then used to access the actual data in RAM, completing the translation process.

PAGES: In paged memory management each process address space is divided into equal pieces called pages.

FRAMES: Physical memory is divided into pieces of the same size called Frames.

Segmentation

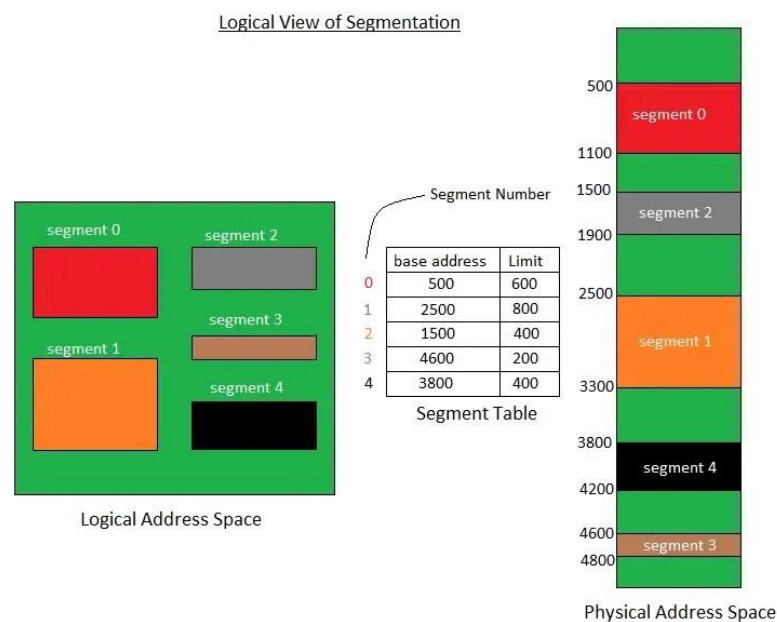
In Operating Systems, Segmentation is a memory management technique in which the memory is divided into the variable size parts. Each part is known as a segment which can be allocated to a process. The details about each segment are stored in a table called a segment table. Segment table is stored in one (or many) of the segments.

Segment table contains mainly two information about segment:

1. **Base:** It is the base address of the segment
2. **Limit:** It is the length of the segment.

How Segmentation Works

1. The OS divides the program into segments.
2. A Segment Table is maintained for each process. It contains:
 - Base (starting address of the segment)
 - Limit (length of the segment)
3. When a program wants to access memory:
 - It provides a segment number and offset (distance).
 - The OS checks if the offset is within the limit.
 - If valid, it adds the base address to the offset to get the physical address.



Advantages of Segmentation

1. No internal fragmentation
2. Average Segment Size is larger than the actual page size.

Disadvantages

1. It can have external fragmentation.
2. Costly memory management algorithms.

Segmentation With Paging

Segmented Paging

- Pure segmentation is not very popular and not being used in many of the operating systems. However, Segmentation can be combined with Paging to get the best features out of both the techniques.

- In Segmented Paging, the main memory is divided into variable size segments which are further divided into fixed size pages.
- Pages are smaller than segments.
- Each Segment has a page table which means every program has multiple page tables.
- The logical address is represented as Segment Number (base address), Page number and page offset.

Segment Number → It points to the appropriate Segment Number.

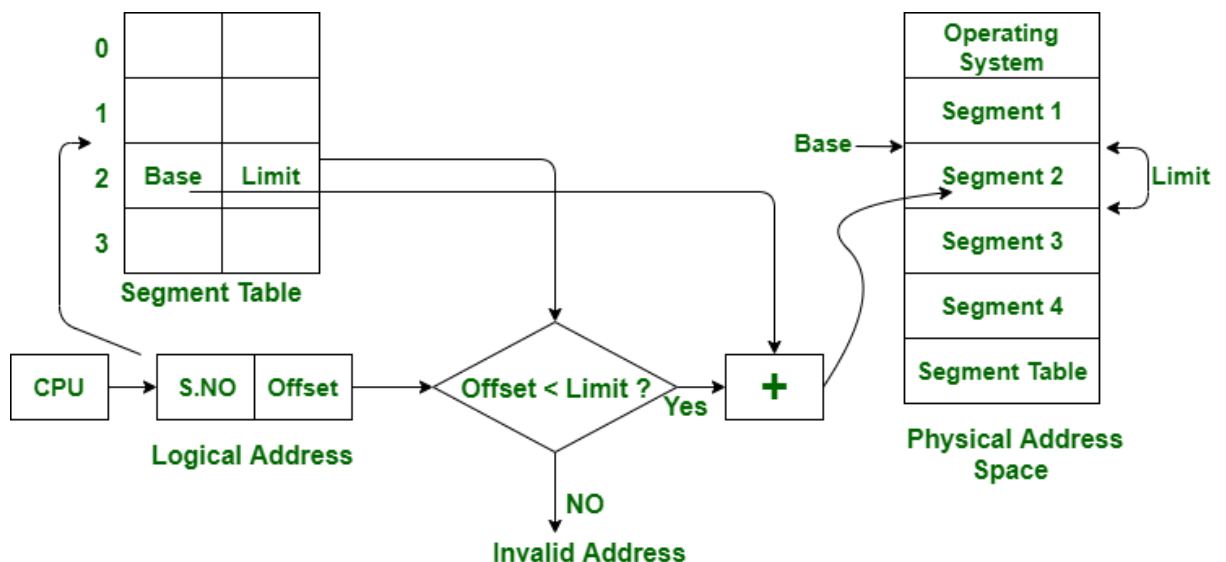
Page Number → It Points to the exact page within the segment

Page Offset → Used as an offset within the page frame

Each Page table contains the various information about every page of the segment. The Segment Table contains the information about every segment. Each segment table entry points to a page table entry and every page table entry is mapped to one of the page within a segment.

Translation of logical address to physical address

- The CPU generates a logical address which is divided into two parts: Segment Number and Segment Offset. The Segment Offset must be less than the segment limit. Offset is further divided into Page number and Page Offset. To map the exact page number in the page table, the page number is added into the page table base.
- The actual frame number with the page offset is mapped to the main memory to get the desired word in the page of the certain segment of the process.



Definition of Virtual Memory

Virtual memory is a memory management technique that allows a computer to use more memory than what is physically available (RAM) by using a part of the hard disk as if it were

RAM. It gives an illusion of a large, continuous memory to the programs, even if the actual physical memory is smaller.

Virtual memory enables multiple programs to run at the same time without running out of RAM and helps protect each program's memory space from others. It improves system efficiency and allows for better multitasking.

Benefits of Virtual Memory

1. **Allows Larger Programs to Run**
Virtual memory enables the system to run programs that are larger than the physical RAM.
2. **Enables Multitasking**
Multiple programs can run at the same time without crashing due to low memory.
3. **Memory Protection**
Each program gets its own memory space, preventing interference with others.
4. **Efficient Memory Use**
Only the necessary parts of a program are loaded into RAM, saving space.
5. **Improves System Stability**
It reduces the chance of system crashes by handling memory overflow using disk space (swap area).

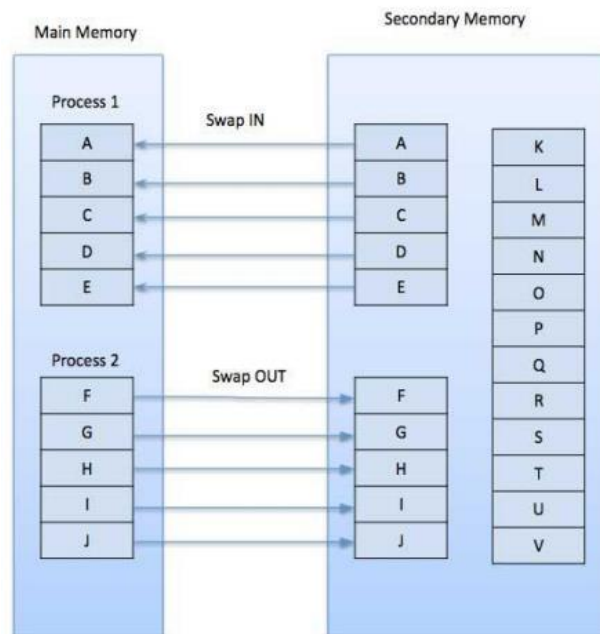
Virtual memory techniques

A computer can address more memory than the amount physically installed on the system. This extra memory is actually called virtual memory and it is a section of a hard disk that's set up to emulate the computer's RAM. Following are the situations, when entire program is not required to be loaded fully in main memory.

- User written error handling routines are used only when an error occurred in the data or computation.
- Certain options and features of a program may be used rarely.
- Many tables are assigned a fixed amount of address space even though only a small amount of the table is actually used.
- The ability to execute a program that is only partially in memory would counter many benefits.
- Less number of I/O would be needed to load or swap each user program into memory.
- A program would no longer be constrained by the amount of physical memory that is available.
- Each user program could take less physical memory, more programs could be run the same time, with a corresponding increase in CPU utilization and throughput.
- Virtual memory is commonly implemented by demand paging. It can also be implemented in a segmentation system. Demand segmentation can also be used to provide virtual memory.

Demand Paging

Demand paging is a memory management technique used in operating systems where a program's pages are loaded into the main memory only when they are actually needed during execution. Instead of loading the entire program into RAM at once, only the required pages are brought in as the program runs. If the program tries to access a page that is not currently in memory, a page fault occurs.



Explanation of the Working:

Main Memory vs Secondary Memory:

- **Main Memory (RAM):** Fast but limited in size.
- **Secondary Memory (Disk):** Slower but large in size.

Step-by-Step Working:

1. Process 1 (Pages A to E) – Swap IN:

- Pages A, B, C, D, E of Process 1 are being swapped IN (moved) from Secondary Memory to Main Memory.
- This means Process 1 is starting or is currently active and its required pages are being loaded into RAM so that it can be executed.

2. Process 2 (Pages F to J) – Swap OUT:

- Pages F, G, H, I, J of Process 2 are being swapped OUT from Main Memory to Secondary Memory.
- This indicates Process 2 is either paused, finished, or not currently active, so its memory space is freed by moving its pages out of RAM.

Swap IN and Swap OUT in Demand Paging:

- **Swap IN:** When a process or page is required and not in memory, it is loaded (swapped in) from disk to RAM.
- **Swap OUT:** When memory is full or a process is inactive, its pages are moved (swapped out) to disk to free up RAM.

Advantages:

1. Saves memory by loading only needed pages.
2. Programs start faster without loading everything.
3. Supports more processes in memory simultaneously.

Disadvantages:

1. Page faults cause delays due to disk access.
2. Adds complexity to OS memory management.
3. Can cause thrashing if memory is overloaded.

Difference between paging and segmentation:

Aspect	Paging	Segmentation
Basic Idea	Divides process (logical memory) into fixed-size blocks called pages .	Divides process into variable-sized segments (like functions, arrays, code, stack, data).
Size	Pages are fixed-size (same for all processes).	Segments are variable-size , depending on logical division of program.
Address Structure	Logical address = Page Number + Page Offset .	Logical address = Segment Number + Segment Offset .
Mapping	Page number → Frame number (via Page Table).	Segment number → Base + Limit (via Segment Table).
Fragmentation	Causes internal fragmentation (unused space inside a page).	Causes external fragmentation (unused holes in memory).
Relation to Program	Pages are not related to program structure , just memory units.	Segments are logical units of program (code, data, stack, etc.).
Ease for OS	Simpler for OS to manage because fixed size.	Harder to manage because of variable size.

Aspect	Paging	Segmentation
Used in	Most modern OS (with demand paging + virtual memory).	Rarely used alone; often combined with paging (segmented paging).
Example	A 4KB page may store half of a function and part of another.	A segment may contain exactly one function or one array.

Page Replacements

Page replacement is a memory management process in operating systems. It occurs when a page fault happens and there is no free memory space. The system selects an existing page in memory to replace with the needed one. Page replacement algorithms like FIFO, LRU, and Optimal guide this selection. Its goal is to reduce page faults and maintain efficient memory usage.

Page Fault – A page fault happens when a running program accesses a memory page that is mapped into the virtual address space, but not loaded in physical memory.

Page Replacement Algorithms

Page replacement algorithms are techniques used in operating systems to manage memory efficiently when the physical memory is full. When a new page needs to be loaded into physical memory, and there is no free space, these algorithms determine which existing page to replace.

There are three basic algorithm are there:

1. First In First Out(FIFO)
2. Least Recently Used(LRU)
3. Optimal Page Replacement

First In First Out (FIFO)

This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

FIFO Rules :

- When a page is requested:
 - If it's **already in memory** → ☒ No page fault.
 - If it's **not in memory** → ☐ Page fault occurs.

- If all frames are full, remove the **oldest loaded page** (the one that came in first).
- Use a **queue** to maintain the order.

Example 1: Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3-page frames. Find the number of page faults using FIFO Page Replacement Algorithm.

Given:

- Page reference string: 1, 3, 0, 3, 5, 6, 3
- Number of page frames: 3

Page reference		1, 3, 0, 3, 5, 6, 3					
1	3	0	3	5	6	3	
		0	0	0	0	3	
	3	3	3	3	6	6	
1	1	1	1	5	5	5	
Miss	Miss	Miss	Hit	Miss	Miss	Miss	

Total Page Fault = 6

Step-by-Step Key Events:

1. Page 1 → Not in memory → Load → **Page Fault**
2. Page 3 → Not in memory → Load → **Page Fault**
3. Page 0 → Not in memory → Load → **Page Fault**
4. Page 3 → Already in memory → **No Page Fault**
5. Page 5 → Not in memory → Remove page 1 (oldest) → Load 5 → **Page Fault**
6. Page 6 → Not in memory → Remove page 3 → Load 6 → **Page Fault**
7. Page 3 → Not in memory → Remove page 0 → Load 3 → **Page Fault**

Final Memory State:

- 5, 6, 3
- Number of Page Faults = 6

Belady's Anomaly

Normally, we expect that if you give more memory (more frames), the system should perform better (i.e., fewer page faults).
But in Belady's Anomaly, adding more frames increases page faults, which is unexpected.

Occurs In:

- Mainly happens in the **FIFO (First In First Out)** page replacement algorithm.
- **Does not occur** in **Optimal** or **LRU** algorithms (they are stack-based and anomaly-free).

Example (classic):

- **Reference String:** 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

With 3 frames:

Results in **9 page faults**

With 4 frames:

Results in **10 page faults** (more faults with more memory)

This is **Belady's Anomaly**

Key Points:

- Belady's Anomaly shows **non-intuitive behavior** in FIFO.
- More memory can sometimes lead to **worse performance**.
- Happens in **FIFO**, not in **LRU** or **Optimal**.

Least Recently Used(LRU)

Least Recently Used (LRU) is a page replacement algorithm used in operating systems that removes the page that has not been used for the longest period of time when a new page needs to be loaded and memory is full.

LRU Algorithm – Rules

1. Track Usage History:
Keep track of when each page was last used.
2. Page Hit:
If the requested page is already in memory, update its last used time.
No page fault.
3. Page Miss (Page Fault):
If the page is not in memory:
 - If there is free space, load the page directly.
 - If memory is full, remove the page that was least recently used (the one not used for the longest time).
 - Load the new page in its place.
4. Always Replace the Oldest Used Page:
Among all pages in memory, choose the one that was used farthest in the past.
5. Maintain Order:
You can use a stack, counter, or list to maintain the order of page usage.

Example Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frames. Find number of page faults using LRU Page Replacement Algorithm.

Given:

- Page reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3
- Number of frames = 4
- Page replacement algorithm: LRU (Least Recently Used)

Page reference	7,0,1,2,0,3,0,4,2,3,0,3,2,3														No. of Page frame - 4	
7	0	1	2	0	3	0	4	2	3	0	3	2	3			
			2	2	2	2	2	2	2	2	2	2	2			
		1	1	1	1	1	4	4	4	4	4	4	4			
	0	0	0	0	0	0	0	0	0	0	0	0	0			
7	7	7	7	7	3	3	3	3	3	3	3	3	3			
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit			
Total Page Fault = 6																

Here LRU has same number of page fault as optimal but it may differ according to question.

Let's trace the algorithm step by step:

- Reference string = 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3
- Number of frames = 4
- Algorithm = LRU (Least Recently Used)

How LRU works

- If the page is already in memory → HIT.
- If not in memory → FAULT.
 - If there is empty space → load it.
 - If frames are full → replace the least recently used page (the one whose last use is farthest in the past).

Step-by-step:

Step 1: Reference = 7

- Frames empty → load 7
 - Frames: [7]
 - Page Fault
-

Step 2: Reference = 0

- Not in frames → load 0
 - Frames: [7, 0]
 - Page Fault
-

Step 3: Reference = 1

- Not in frames → load 1
 - Frames: [7, 0, 1]
 - Page Fault
-

Step 4: Reference = 2

- Not in frames → load 2
 - Frames: [7, 0, 1, 2] (all 4 full now)
 - Page Fault
-

Step 5: Reference = 0

- 0 is already in frames
 - Just update “recently used” info
 - Frames: [7, 0, 1, 2]
 - Hit
-

Step 6: Reference = 3

- 3 is not in frames
 - Must evict LRU among [7, 0, 1, 2]
 - Last use of 7 → Step 1
 - Last use of 0 → Step 5
 - Last use of 1 → Step 3
 - Last use of 2 → Step 4
 - 7 is LRU → evict 7, insert 3
 - Frames: [3, 0, 1, 2]
 - Page Fault
-

Step 7: Reference = 0

- 0 is in frames

- Update usage
 - Frames: [3, 0, 1, 2]
 - Hit
-

Step 8: Reference = 4

- 4 is not in frames
 - Must evict LRU among [3, 0, 1, 2]
 - Last use of 3 → Step 6
 - Last use of 0 → Step 7
 - Last use of 1 → Step 3
 - Last use of 2 → Step 4
 - 1 is oldest (Step 3) → evict 1, insert 4
 - Frames: [3, 0, 4, 2]
 - Page Fault
-

Step 9: Reference = 2

- 2 is in frames
 - Update usage
 - Frames: [3, 0, 4, 2]
 - Hit
-

Step 10: Reference = 3

- 3 is in frames
 - Update usage
 - Frames: [3, 0, 4, 2]
 - Hit
-

Step 11: Reference = 0

- 0 is in frames
 - Update usage
 - Frames: [3, 0, 4, 2]
 - Hit
-

Step 12: Reference = 3

- 3 is in frames

- Update usage
 - Frames: [3, 0, 4, 2]
 - Hit
-

Step 13: Reference = 2

- 2 is in frames
 - Update usage
 - Frames: [3, 0, 4, 2]
 - Hit
-

Step 14: Reference = 3

- 3 is in frames
 - Update usage
 - Frames: [3, 0, 4, 2]
 - Hit
-

Final Results:

- Total references: 14
- Page Faults: 6 (steps: 1, 2, 3, 4, 6, 8)
- Page Hits: 8
- Evictions: 7 → 1

Optimal Page Replacement

The Optimal Page Replacement algorithm is a theoretical page replacement algorithm used in operating systems for managing virtual memory. Its core principle is to replace the page currently in memory that will not be used for the longest period in the future.

Rules of Optimal Page Replacement:

1. **Know the future:**
 - At each page fault, look ahead in the reference string.
 - Identify which of the pages in memory will not be used for the longest time.
2. **Replacement decision:**
 - If the needed page is not in memory (page fault), replace the page that won't be used for the longest time from now onward.
3. **If a page will never be used again**, it is the best candidate to be replaced.

Example: Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frame. Find number of page fault using Optimal Page Replacement Algorithm.

Given :

- **Page Reference String:** 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3
- **Number of Page Frames:** 4

Page reference	7,0,1,2,0,3,0,4,2,3,0,3,2,3														No. of Page frame - 4													
	7	0	1	2	0	3	0	4	2	3	0	3	2	3														
				2	2	2	2	2	2	2	2	2	2	2														
			1	1	1	1	1	4	4	4	4	4	4	4														
		0	0	0	0	0	0	0	0	0	0	0	0	0														
	7	7	7	7	7	3	3	3	3	3	3	3	3	3														
	Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit														
Total Page Fault = 6																												

Step-by-step execution

Step 1: Reference = 7

- Frames: empty → load 7.
 - **Page fault.**
 - Frames = [7]
-

Step 2: Reference = 0

- Not in frames → load 0.
 - **Page fault.**
 - Frames = [7, 0]
-

Step 3: Reference = 1

- Not in frames → load 1.
 - **Page fault.**
 - Frames = [7, 0, 1]
-

Step 4: Reference = 2

- Not in frames → load 2.
- **Page fault.**

- Frames = [7, 0, 1, 2] (now full)
-

Step 5: Reference = 0

- Already in frames.
 - **Page hit.**
 - Frames = [7, 0, 1, 2]
-

Step 6: Reference = 3

- Not in frames → must evict one.
 - Look ahead:
 - 7 → **never used again**
 - 0 → used soon
 - 1 → never used again
 - 2 → used later
 - Optimal → evict 7 (farthest future use = never).
 - **Page fault.**
 - Frames = [3, 0, 1, 2]
-

Step 7: Reference = 0

- Already in frames.
 - **Page hit.**
 - Frames = [3, 0, 1, 2]
-

Step 8: Reference = 4

- Not in frames → must evict one.
 - Look ahead:
 - 3 → used at step 10
 - 0 → used at step 11
 - 1 → **never used again**
 - 2 → used at step 9
 - Optimal → evict 1.
 - **Page fault.**
 - Frames = [3, 0, 4, 2]
-

Step 9: Reference = 2

- Already in frames.
 - **Page hit.**
 - Frames = [3, 0, 4, 2]
-

Step 10: Reference = 3

- Already in frames.
 - **Page hit.**
 - Frames = [3, 0, 4, 2]
-

Step 11: Reference = 0

- Already in frames.
 - **Page hit.**
 - Frames = [3, 0, 4, 2]
-

Step 12: Reference = 3

- Already in frames.
 - **Page hit.**
 - Frames = [3, 0, 4, 2]
-

Step 13: Reference = 2

- Already in frames.
 - **Page hit.**
 - Frames = [3, 0, 4, 2]
-

Step 14: Reference = 3

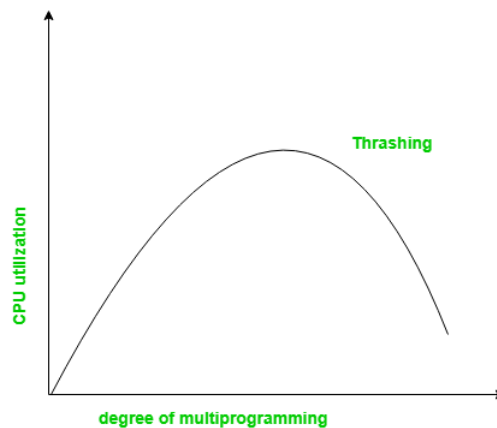
- Already in frames.
 - **Page hit.**
 - Frames = [3, 0, 4, 2]
-

 Final Result

- **Total Page Faults** = 6 (steps 1, 2, 3, 4, 6, 8)
- **Total Page Hits** = 8

Concept Of Thrashing

Thrashing is a situation in a virtual memory system where the CPU spends more time swapping pages in and out of memory (paging) than executing actual processes. This results in severe performance degradation.



Why Does Thrashing Occur?

Thrashing happens when:

1. Too many processes are loaded into memory.
2. Each process does not have enough frames to hold its working set (the set of pages it is actively using).
3. As a result, processes continuously generate page faults.
4. The OS constantly swaps pages in and out (page replacement), trying to satisfy these faults, which leads to more faults.
5. The system becomes overloaded with paging activity and does little useful work.

Causes of thrashing:

- Too many processes in memory (high multiprogramming).
- Insufficient physical memory for active processes.
- Poor page replacement algorithms replacing useful pages.
- Large working sets that exceed allocated memory.
- Frequent context switching causing repeated page faults.

Working Set Model

The **Working Set Model** is a memory management strategy used to reduce page faults and **prevent thrashing** by keeping the most-needed pages in memory.

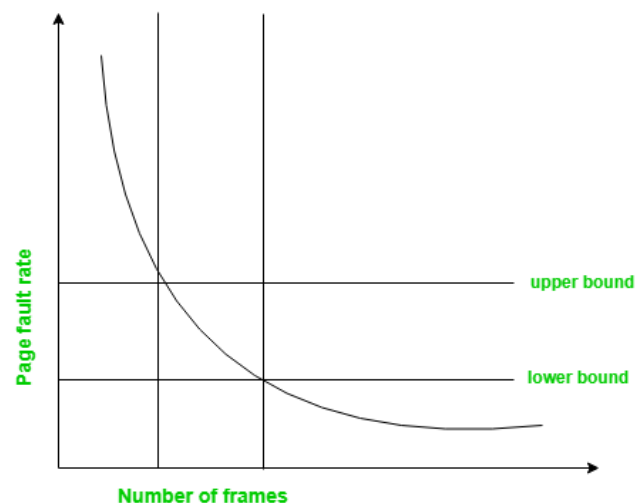
1. Based on the **Locality Model** — processes use memory in localized patterns.
2. If a process is given enough frames to fit its current **locality**, page faults will be minimal.
3. If given **fewer frames** than its locality size, the process will **thrash**.
4. The **working set** is defined by the most recent '**A**' **page references**.
5. The value of **A** affects accuracy:
 - Large **A** → overlapping working sets.
 - Small **A** → may miss entire locality.
6. Let **D** be the total frame demand (sum of working set sizes), and **m** the available frames:
 - If **D > m**, thrashing occurs.
 - If **D ≤ m**, no thrashing.

Benefits of the Working Set Model:

- Reduces page faults.
- Prevents thrashing by tracking actual memory needs.
- Helps the OS decide how many processes to keep in memory.

Page Fault Frequency

The Page Fault Frequency (PFF) algorithm is a memory management strategy used to control thrashing by monitoring and adjusting how often page faults occur.



1. Thrashing is caused by a high page fault rate, so PFF aims to control it.
2. A high page fault rate means too few frames are allocated to the process.
3. A low page fault rate means the process may have more frames than needed.
4. Upper and lower thresholds are set for acceptable page fault rate.
5. If the rate exceeds the upper limit, allocate more frames to the process.
6. If the rate falls below the lower limit, remove some frames to optimize usage.
7. If no free frames are available, suspend some processes and reallocate their frames temporarily.

Benefits of Page Fault Frequency (PFF)

1. **Reduces thrashing** by dynamically adjusting memory allocation.
2. **Improves system performance** by keeping the page fault rate within acceptable limits.
3. **Efficient use of memory** — avoids over- or under-allocation of frames.
4. **Adapts to changing process needs**, unlike static allocation methods.
5. **Minimizes unnecessary swapping**, reducing disk I/O overhead.